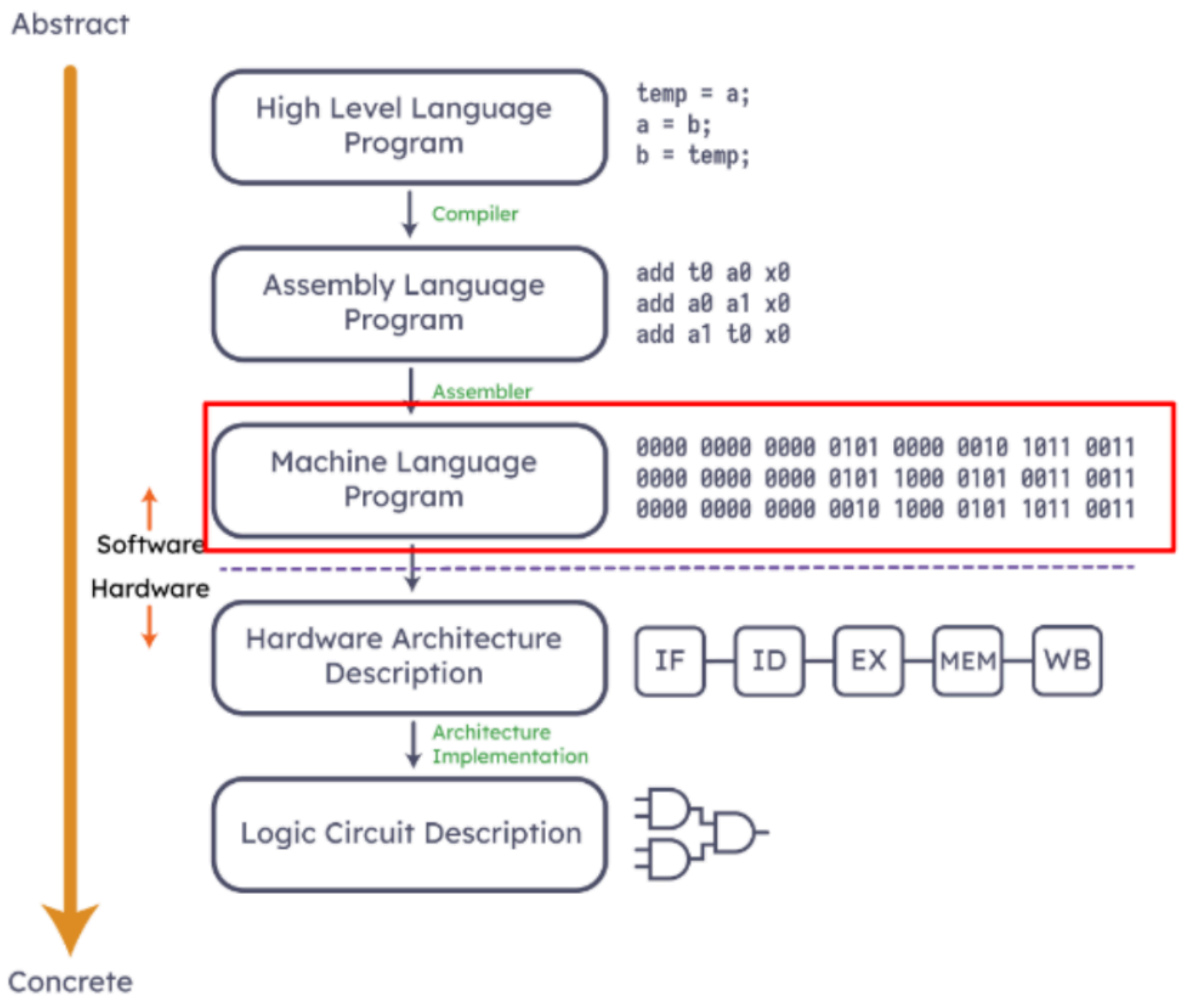
219223 Computer System Laboratory

Lab 7-2: RISC-V machine code (1)

Make a copy of this file and put your answers to the questions posted in this lab there.

Machine code layer



RISC-V machine code is always 4-byte long and is a binary representation of each RISC-V assembly instruction

R-format instructions

```
add  x18,x19,x10
```

31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0
funct7	rs2	rs1	funct3	rd	opcode
7	5	5	3	5	7
0000000	01010	10011	000	10010	0110011
add	rs2=10	rs1=19	add	rd=18	Reg-Reg OP

Involve 3 registers

- rs1 - source operand 1
- rs2 - source operand 2
- rd - destination

The following table shows opcode, funct3, and funct7 for other R-format instructions

Instruction	Name	Description	Type	Opcode	Funct3	Funct7
add rd rs1 rs2	ADD	rd = rs1 + rs2	R	011 0011	000	000 0000
sub rd rs1 rs2	SUBtract	rd = rs1 - rs2	R	011 0011	000	010 0000
and rd rs1 rs2	bitwise AND	rd = rs1 & rs2	R	011 0011	111	000 0000
or rd rs1 rs2	bitwise OR	rd = rs1 | rs2	R	011 0011	110	000 0000
xor rd rs1 rs2	bitwise XOR	rd = rs1 ^ rs2	R	011 0011	100	000 0000
sll rd rs1 rs2	Shift Left Logical	rd = rs1 << rs2	R	011 0011	001	000 0000
srl rd rs1 rs2	Shift Right Logical	rd = rs1 >> rs2 (Zero-extend)	R	011 0011	101	000 0000
sra rd rs1 rs2	Shift Right Arithmetic	rd = rs1 >> rs2 (Sign-extend)	R	011 0011	101	010 0000
slt rd rs1 rs2	Set Less Than (signed)	rd = (rs1 < rs2) ? 1 : 0	R	011 0011	010	000 0000
sltu rd rs1 rs2	Set Less Than (Unsigned)		R	011 0011	011	000 0000

<u>Answer the following questions</u>

Translate the following RISC-V assembly into machine code. Answer in hexadecimal form. You must show your work to receive credits.

- `add x4, x3, x2`

funct7	rs2	rs1	funct3	rd	opcode
0000000	00010	00011	000	00100	0110011
0000 000	0 0010	0001 1	000	0010 0	011 0011

Binary -> Value

0000	0000	0010	0001	1000	0010	0011	0011

0	0	2	1	8	2	3	3

Answer: 0x00218233

- sll x10, x11, x12

funct7	rs2	rs1	funct3	rd	opcode
0000000	01100	01011	001	01010	0110011
0000 000	0 1100	0101 1	001	0101 0	011 0011

Binary -> Value

0000	0000	1100	0101	1001	0101	0011	0011
0	0	C	5	9	5	3	3

Answer: 0x00C59533

```
opcode: 0110011
rd: 01010
funct3: 001
rs1: 01011
rs2: 01100
funct7: 0000000

  0000 0000 1100 0101 1001 0101 0011 0011
```

0x00C59533

Need refreshment on converting from binary to hexadecimal, play this game:

https://flippybitandtheattackofthehexadecimalsfrombase16.com/

- slt x22, x22, x23

funct7	rs2	rs1	funct3	rd	opcode
0000000	10111	10110	010	10110	0110011
0000 000	1 0111	1011 0	010	1011 0	011 0011

Binary -> Value

0000	0001	0111	1011	0010	1011	0011	0011
0	1	7	B	2	B	3	3

Answer: 0x017B2B33

I-format instructions

Arithmetic/logic

```
addi  x15,x1,-50
```

31 … 20	19 … 15	14 … 12	11 … 7	6 … 0
imm[11:0]	rs1	funct3	rd	opcode
12	5	3	5	7

111111001110	00001	000	01111	0010011
imm=-50	rs1=1	add	rd=15	OP-Imm

Arithmetic/logic Instructions involving an immediate operand as per the above example

The following table shows opcode, funct3, and funct7 for other I-format: arithmetic/logic instructions

funct3

opcode

imm[11:0]		rs1	000	rd	0010011	addi
imm[11:0]		rs1	010	rd	0010011	slti
imm[11:0]		rs1	011	rd	0010011	sltiu
imm[11:0]		rs1	100	rd	0010011	xori
imm[11:0]		rs1	110	rd	0010011	ori
imm[11:0]		rs1	111	rd	0010011	andi
0000000	shamt	rs1	001	rd	0010011	slli
0000000	shamt	rs1	101	rd	0010011	srli
0100000	shamt	rs1	101	rd	0010011	srai

This bit position is used to distinguish srli and srai

shamt only needs 5 bits as a shift amount in a 32-bit system can never be more than 31.

<u>Answer the following questions</u>
Translate the following RISC-V assembly into machine code. Answer in hexadecimal form. You must show your work to receive credits.

- `addi x4, x10, 24`

imm[11:0]	rs1	funct3	rd	opcode
000000011000	01010	000	00100	0010011
0000 0001 1000	0101 0	000	0010 0	001 0011

Binary -> Value

0000	0001	1000	0101	0000	0010	0001	0011
0	1	8	5	0	2	1	3

Answer: 0x01850213

- `ori x10, x11, 0xc7` (1100 0111)

imm[11:0]	rs1	funct3	rd	opcode
0xc7	01011	110	01010	0010011

0xc7	0101 1	110	0101 0	001 0011

Binary -> Value

0	c	7	0101	1110	0101	0001	0011
0	c	7	5	E	5	1	3

Answer: 0x0C75E513

- `srai x22, x18, 17`

imm[11:5]	imm[4:0]	rs1	funct3	rd	opcode
0100000	10001	10010	101	10110	0010011
0100 000	1 0001	1001 0	101	1011 0	001 0011

Binary -> Value

0100	0001	0001	1001	0101	1011	0001	0011
4	1	1	9	5	B	1	3

Answer: 0x41195B13

I-format: load

```
lw x14, 8(x2)
```

31 – 20	19 – 15	14 – 12	11 – 7	6 – 0
imm[11:0]	rs1	funct3	rd	opcode
12	5	3	5	7
offset[11:0]	base	width	dest	LOAD
000000001000	00010	010	01110	0000011
imm=+8	rs1=2	lw	rd=14	LOAD
		(load word)		

The following table shows opcode, funct3, and funct7 for other I-format: load instructions

				opcode	funct3
lb rd imm(rs1)	Load Byte	rd = 1 byte of memory at address rs1 + imm, sign-extended	I	000 0011	000
lbu rd imm(rs1)	Load Byte (Unsigned)	rd = 1 byte of memory at address rs1 + imm, zero-extended	I	000 0011	100
lh rd imm(rs1)	Load Half-word	rd = 2 bytes of memory starting at address rs1 + imm, sign-extended	I	000 0011	001
lhu rd imm(rs1)	Load Half-word (Unsigned)	rd = 2 bytes of memory starting at address rs1 + imm, zero-extended	I	000 0011	101
lw rd imm(rs1)	Load Word	rd = 4 bytes of memory starting at address rs1 + imm	I	000 0011	010

<u>Answer the following questions</u>

Translate the following RISC-V assembly into machine code. Answer in hexadecimal form. You must show your work to receive credits.

- `lw x22, 12(x1)`

imm[11:0]	rs1	funct3	rd	opcode
000000001100	00001	010	10110	0000011
0000 0000 1100	0000 1	010	1011 0	000 0011

Binary -> Value

0000	0000	1100	0000	1010	1011	0000	0011
0	0	C	0	A	B	0	3

Answer: 0x00C0AB03

- `lb x22, 12(x1)`

imm[11:0]	rs1	funct3	rd	opcode
000000001100	00001	000	10110	0000011
0000 0000 1100	0000 1	000	1011 0	000 0011

Binary -> Value

0000	0000	1100	0000	1000	1011	0000	0011
0	0	C	0	8	B	0	3

Answer: 0x00C08B03

- `lhu x19, 0(x5)`

imm[11:0]	rs1	funct3	rd	opcode
000000000000	00101	101	10011	0000011
0000 0000 0000	0010 1	101	1001 1	000 0011

Binary -> Value

0000	0000	0000	0010	1101	1001	1000	0011

0	0	0	2	D	9	8	3

Answer: 0x0002D983

S-format instructions

sw x14, 8(x2)

31 – 25	24 – 20	19 – 15	14 – 12	11 – 7	6 – 0
Imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
7	5	5	3	5	7
offset[11:5]	src	base	width	offset[4:0]	STORE
0000000	01110	00010	010	01000	0100011
offset[11:5]	rs2=14	rs1=2	SW	offset[4:0]	STORE

This is the format used exclusively for store-type instructions.

The following table shows opcode, funct3, and funct7 for other store instructions.

			funct3		opcode	
Imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	sb
Imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	sh
Imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	sw

<u>Answer the following questions</u>
Translate the following RISC-V assembly into machine code. Answer in hexadecimal form. You must show your work to receive credits.

- `sw x14, 8(x1)`

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
0000000	01110	00001	010	01000	0100011
0000 000	0 1110	0000 1	010	0100 0	010 0011

Binary -> Value

0000	0000	1110	0000	1010	0100	0010	0011
0	0	E	0	A	4	2	3

Answer: 0x00E0A423

- `sb x22, 12(x19)`

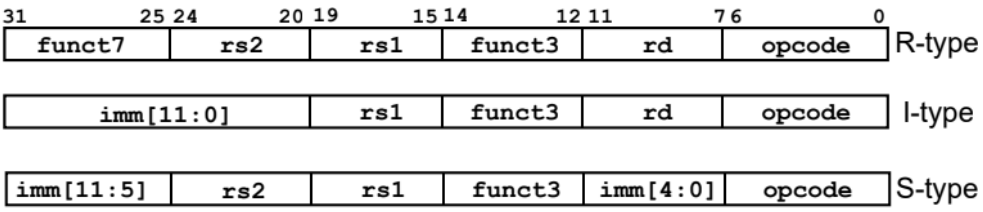
imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
0000000	10110	10011	000	01100	0100011
0000 000	1 0110	1001 1	000	0110 0	010 0011

Binary -> Value

0000	0001	0110	1001	1000	0110	0010	0011
0	1	6	9	8	6	2	3

Answer: 0x01698623

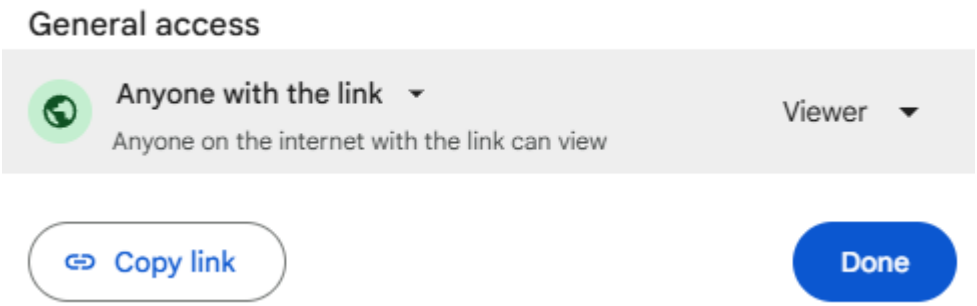
In summary, here is what we have learned about RISC-V machine code so far.



Submission

- Click “Share” on the top right corner of the document you put all your answer in

- Go to “General access” and select “Anyone with the link”. Then, click “Copy link” and “Done”



- Send the link copied to Google Classroom for grading